SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA0612 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15 **Total Marks:25**

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I K. Pujitha (192524149) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student*: K. Pujitha*

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability. (5 Marks)

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**30**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**10**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Perceptron fails to converge

A perceptron may fail to converge even on linearly separable data due to incorrect learning rate, poor weight initialization, bias errors or mistakes in the update rule.

Debugging and Optimization :-

1) Check data: Verify that the dataset is correctly labelled and linearly separable.
2) Learning Rate: choose a suitable learning rate to avoid slow or unstable learning.
3) weight initialization: Initialize weights with small values and set bias correctly.
4) Update rule: verify the weight update formula:

$$w_{new} = w_{old} + \eta (y - \hat{y}) x$$

5) Test and optimize:- check predictions after each Epoch, monitor errors, & stop when all samples are classified correctly.

Conclusion:

Correct implementation and suitable parameter selection help the perceptron converge faster and more reliably.

2) Slow Convergence and local minima in MLP

A multilayer perceptron may learn slowly due to vanishing gradients, unsuitable activation functions, Poor weight initialization, or an inappropriate learning rate.

Debugging and optimization:

1. check the training and validation loss to identify learning problems.
2. Use ReLU in hidden layers to reduce the vanishing gradient problem.
3. Initialize weights properly using Xavier or He initialization
4. Normalize input features to improve stable learning.
5. Use adaptive optimizers such as Adam & adjust the learning rate.

Conclusion:

Proper initialization, normalization, suitable activation functions, and optimized learning rates improve convergence and accuracy.

Genetic Algorithm produces inconsistent solutions :-

A Genetic Algorithm may produce Poor solutions due to incorrect chromosome Ending, an unsuitable fitness function, premature Convergence or insufficient Population diversity.

Debbuging and optimization:

1) Encoding :- Verify that chromosomes Correctly represent Candidate solutions.
2) Fitness function: Ensure that fitness accurately measures solution quality.
3) mutation: Adjust the mutation rate to introduce diversity and Explore new solutions.
4) selection pressure: Avoid selecting only the best individuals, as this reduces diversity.
5) Elitism: Preserve a few best individuals in Each Generation to prevent losing good solutions.

Conclusion:

Proper encoding, fitness evaluation, mutation and selection improve solution quality losing good solutions.

4) Deep Neural Network with Genetic Programming

A deep neural network combined with Genetic Programming may suffer from overfitting, high computational cost, excessive model complexity, and insufficient training data.

Debugging and optimization:

1) Examine training and validation performance to identify overfitting.
2) Use regularization, such as L1 or L2, to Reduce overfitting.
3) Apply dropout to prevent excessive dependence on Particular neurons.
4) Use pruning to remove unnecessary connections or model components.
5) Reduce model complexity, use sufficient training data, and Evaluate performance using validation data.

Conclusion:

Regularization, dropout, Purning and Efficient Evaluation improve generalization while reducing computational cost.

Decision Tree overfits the training data

A Decision Tree may achieve high training accuracy but Poor test accuracy because it is too deep, learns noise, or contains irrelevant features.

Debugging and optimization:

1) Check overfitting: Compare training accuracy with validation accuracy.
2) Pruning: Remove unnecessary branches to simplify the tree.
3) Limit tree depth :- Set maximum depth and minimum samples per leaf.
4) Cross-validation: Evaluate the model on different data splits to select suitable Parameters.
5) Feature selection: Remove irrelevant features and handle noisy or missing data.

Conclusion:

Purning, cross-validation and feature selection reduce overfitting and improve the decision tree's performance on unseen data.